

Native Packaging and App Update Guidance

Last updated: June 4, 2026

This private/personal app is currently treated as a PWA-first release. Native packaging can be added later if there is a clear need for app-store distribution, native notification channels, or managed mobile deployment. Do not hardcode secrets in any wrapper. Supabase service role keys, private VAPID keys, database credentials, and signing credentials must never ship in client or native app bundles.

Current Release Model

- Primary release path: Vercel-hosted PWA.
 - Install path: browser install prompt, Android Chrome install, or iPhone/iPad Safari "Add to Home Screen".
 - Updates: deployed through Vercel. PWA users receive new web assets as the service worker/browser cache updates.
 - Push notifications: Web Push through the existing service worker and VAPID configuration.
 - In-app sounds: category-based browser audio only where user interaction and browser policy permit.
-

Android Packaging Options

Trusted Web Activity

Trusted Web Activity is the lowest-friction Android option for a PWA-style app.

- Uses Chrome Custom Tabs technology to present the production web app.
- Requires Digital Asset Links between the Android package and the production domain.
- Keeps most app logic in the deployed web app.
- Works well when the app is already PWA-ready.
- Native features beyond the web platform require additional native code.

Capacitor

Capacitor is a stronger wrapper if future native APIs are needed.

- Can package the app inside an Android project.

- Can bridge native APIs such as notification channels, local notifications, secure storage, and device APIs.
- Requires maintaining native project files, signing config, app permissions, and store builds.
- Any Supabase or service credentials inside native code must be public/client-safe only.

Other Wrapper Options

- Bubblewrap can generate a TWA wrapper from the PWA manifest.
 - A fully custom Android wrapper is possible but should be avoided unless there is a clear native requirement.
 - Do not duplicate the web notification, auth, upload, or setup systems inside the wrapper.
-

Android Package Name Strategy

- Pick a stable reverse-DNS package name before the first Play Console upload.
 - Example pattern: `com.yourdomain.caregiverapp`.
 - Do not change the package name after launch unless creating a separate app listing.
 - Keep private/personal builds separate from any public Carer Vista Pro package.
 - Document the package name in release notes and signing docs.
-

Android Signing Key Strategy

- Prefer Play App Signing for Play Store releases.
 - Keep the upload key outside the repository.
 - Store key ownership and recovery information in a secure password manager.
 - Never commit `.jks`, `.keystore`, passwords, signing config with secrets, or CI signing secrets.
 - Document who owns the key and who can rotate/recover it.
-

Android Version Rules

- `versionCode` must increase for every uploaded Android build.
- `versionName` is the human-readable release version.
- Suggested convention:
 - `versionName`: `major.minor.patch`, such as `1.0.0`.
 - `versionCode`: monotonically increasing integer, such as `10000`, `10001`.

- Web app package version and native app version should be tracked together in release notes.
-

Android AAB Release Workflow

1. Build the Android wrapper from the production web URL or bundled web assets.
2. Verify package name, app name, icons, permissions, and version values.
3. Build a signed Android App Bundle (`.aab`).
4. Upload to Play Console.
5. Create a release in the target track.
6. Add release notes.
7. Review warnings, policy declarations, Data Safety, and permissions.
8. Roll out to internal testing first.
9. Promote to closed testing if needed.
10. Promote to production with staged rollout.

Google Play releases are prepared in Play Console by creating a release, uploading an app bundle, adding release notes, reviewing, then rolling out.

Android Testing Tracks

- Internal testing: first smoke test for install, login, push, app icons, and navigation.
 - Closed testing: family/caregiver test group before production.
 - Production staged rollout: start small, such as 5% or 10%, then increase after logs and feedback are clean.
 - Pause rollout if login, push, check-in/out, document access, or emergency access regressions appear.
-

Android Play Console Checklist

- ☐ App bundle uploaded.
- ☐ Internal testing track passes.
- ☐ Closed testing track passes if used.
- ☐ Production staged rollout percentage selected.
- ☐ Release notes entered.
- ☐ Play Console Data Safety reviewed.

- ☐ Privacy policy URL set.
 - ☐ Account deletion/support process documented.
 - ☐ App category selected.
 - ☐ No unsupported medical or emergency-service claims.
 - ☐ Adaptive icon verified.
 - ☐ Maskable icon verified.
 - ☐ Push notification disclosure accurate.
-

Android Update Behavior

- Auto updates are controlled by the user, device, and Play Store settings.
 - The app cannot force Play Store auto updates for everyone.
 - The app can show its own "Update available" banner based on a server-side version manifest.
 - The banner can link to the Play Store listing or reload the PWA, depending on packaging model.
 - Critical notices must not block emergency access.
-

Apple Packaging

Native iOS distribution requires the Apple Developer Program.

Bundle ID Strategy

- Reserve a stable Bundle ID before TestFlight.
- Example pattern: `com.yourdomain.caregiverapp`.
- Keep personal/private iOS bundle IDs separate from any public Carer Vista Pro bundle.
- Do not change Bundle ID after launch unless creating a separate app.

Xcode or Wrapper Strategy

- Use Capacitor or another maintained wrapper if native iOS packaging is needed.
- Keep the production URL/config outside committed secrets.
- Do not embed service role keys, private VAPID keys, or database secrets.
- Validate App Transport Security and allowed domains.

Apple Version Rules

- `CFBundleShortVersionString` is the public version, such as `1.0.0`.

- `CFBundleVersion` is the build number and must increase for each uploaded build.
 - App Store Connect requires a new app version with an incremental version number and a new uploaded build for updates.
 - There is no rollback to an old App Store version without submitting a new version.
-

TestFlight Plan

- Create an internal TestFlight group for owner/admin testing.
 - Add external testers only after internal smoke testing passes.
 - Test login, setup, schedule, documents, photos, push, PWA/native differences, emergency access, and legal/help links.
 - Keep release notes clear and short.
 - Include review notes that the app is a personal care coordination and recordkeeping tool, not emergency dispatch or medical advice.
-

App Store Connect Metadata

- App name.
 - Subtitle/description.
 - Keywords.
 - Support URL.
 - Privacy policy URL.
 - Screenshots.
 - App category.
 - Privacy details.
 - Age rating.
 - Review notes and test account if needed.
-

Apple Phased Release

- Apple supports phased release over 7 days for eligible updates.
- Auto updates are controlled by users, devices, and App Store behavior.
- The app can show a "What's new" or "Update available" message using an app version manifest.
- iOS updates still go through the App Store.

Push Notifications and Native Channels

Current PWA Behavior

- Web Push uses the existing service worker.
- Category preferences are stored in the app.
- In-app tones can play only after browser audio is allowed.
- PWA notifications do not guarantee phone-level per-category sounds.

Future Native Android

Use Android notification channels by category:

- `messages` : Messages.
- `shifts` : Shift updates.
- `urgent` : Urgent/emergency alerts.
- `documents` : Documents and print approvals.
- `invoices` : Payments and invoices.
- `reminders` : Reminders.

Important Android channel behavior:

- Users control channel sound, vibration, badges, and visibility in system settings.
- Once channels are created, behavior cannot be freely changed programmatically.
- To materially change channel defaults, use a new channel ID and migration plan.
- Urgent channels should be clear but must not claim emergency dispatch.

Future Native iOS

- Use APNs for native push if a native wrapper is built.
- Use iOS notification categories for actions/grouping where useful.
- Custom notification sounds require bundled native sound files and APNs payload support.
- User and system settings still control final notification behavior.
- Do not promise guaranteed audible alerts.

In-App Update Notification Design

The app can control its own update notice independently of Play Store/App Store auto-update behavior.

Version Manifest

Use a server-side or static version manifest such as:

```
{
  "latestVersion": "1.0.1",
  "minimumSupportedVersion": "1.0.0",
  "critical": false,
  "releaseDate": "2026-06-04",
  "storeUrl": "https://example.com/app",
  "notes": [
    "Improved document access.",
    "Updated notification diagnostics."
  ]
}
```

Implementation options:

- Static file at `/app-version.json`.
- Admin-managed database row.
- Version endpoint backed by app config.

Client Behavior

- Store current app version in package/app config.
- Check latest version on startup and when opening Help/About.
- If newer version exists, show:
 - "Update available"
 - "What's new"
 - "Update now"
 - "Later"
- Store dismissed version in `localStorage` or user preference so users are not spammed.
- Show release notes once after app version changes.
- Critical update notices can be stronger but must not block emergency access.

Future Native Android

- Document Play Core In-App Updates as a future option if the native wrapper supports it.
- Flexible updates are less disruptive.
- Immediate updates should be reserved for severe issues and still respect emergency access requirements.
- Android notification channels can include update notices if appropriate.

Future Native iOS

- Link users to the App Store update page.
 - Use the same version manifest for "Update available" and "What's new".
 - Do not promise automatic forced updates.
-

What's New System

Recommended lightweight design:

- Store latest release notes in a repo file, static JSON manifest, or admin setting.
 - Include version, date, summary, and bullet notes.
 - Show once after app version changes.
 - Store dismissed version per user/device.
 - Add a "View release notes" link in Help/About when a public Help/About release section exists.
 - Do not show repeatedly after dismissal.
-

Release Notes Template

```
## Version 1.0.1 - 2026-06-04

### Fixed
- Improved push notification diagnostics.
- Fixed private document display.

### Changed
- Added release readiness checklist.

### Notes
- No database reset required.
- No user data deletion required.
```

Security Rules

- Do not hardcode secrets.
- Do not ship service role keys.
- Do not ship private VAPID keys.
- Do not commit signing keys.

- Do not commit `.env` files.
 - Do not commit `supabase/.temp`.
 - Keep native wrapper config environment-specific.
 - Keep private bucket media behind signed/authenticated access.
-

Final Native Packaging Readiness Checklist

- ☐ Production web deployment passes smoke tests.
- ☐ Native wrapper option selected.
- ☐ Package name/bundle ID selected.
- ☐ Signing strategy documented.
- ☐ Versioning strategy documented.
- ☐ Icons and splash assets verified.
- ☐ Push behavior documented.
- ☐ In-app update notice design approved.
- ☐ Release notes process documented.
- ☐ Privacy policy/support/deletion URLs verified.
- ☐ Emergency and medical disclaimers reviewed.
- ☐ No secrets in native code or repository.